

1. Perform version control using git/github- install git, create github account, initialize git repository, make commits, creating and merging branches (using CLI/GUI).

1. Installing Git

For Windows

1. Download Git from the official website: <https://git-scm.com/downloads>
2. Run the installer and follow the installation instructions.
3. Verify the installation by opening the command prompt and running:

```
git -version
```

2. Creating a GitHub Account

1. Go to <https://github.com/>
2. Click on **Sign up**.
3. Enter your email, username, and password.
4. Follow the on-screen instructions to complete the setup.
5. Verify your email to activate your GitHub account.

3. Configuring Git

After installation, configure Git with your GitHub credentials:

```
git config --global user.name "Your Name"  
git config --global user.email your-email@example.com
```

To check your configuration:

```
git config -list
```

4. Initializing a Git Repository

1. Create a new folder for your project

```
mkdir my_project  
cd my_project
```

2. Initialize Git in the directory:

```
git init
```

3. Check the status of the repository:

```
git status
```

5. Making Commits

1. Create a new file:

```
echo "Hello, Git!" > file.txt
```

2. Add the file to the staging area:

```
git add file.txt
```

3. Commit the changes:

```
git commit -m "Initial commit"
```

4. View commit history:

```
git log
```

6. Creating and Merging Branches

Creating a New Branch

1. Create a new branch named `feature-branch`:

```
git branch feature-branch
```

2. Switch to the new branch:

```
git checkout feature-branch
```

3. Make changes and commit:

```
echo "New feature" > feature.txt  
git add feature.txt  
git commit -m "Added a new feature"
```

Merging Branches

1. Switch back to the `main` branch:

```
git checkout main
```

2. Merge the `feature-branch` into `main`:

```
git merge feature-branch
```

3. Delete the merged branch:

```
git branch -d feature-branch
```

7. Pushing to GitHub

1. Create a repository on GitHub.

2. Link your local repository to GitHub:

```
git remote add origin https://github.com/yourusername/my_project.git
```

3. Push changes to GitHub:

```
git push -u origin main
```

2. To perform version control using github integration and collaboration- Push local commits to github and manage pull request, fork a repository make changes to forked repo and merge the changes.

Pushing Local Commits to GitHub

Step 1: Create a Repository on GitHub

- Go to GitHub and create a new repository (do not initialize with a README).

Step 2: Link the Local Repository to GitHub

```
git remote add origin https://github.com/your-username/your-repo.git
```

```
git branch -M main
git push -u origin main
```

Step 3: Verify Remote Repositories

```
git remote -v
```

Managing Pull Requests and Collaborations

Step 1: Clone a Repository

```
git clone https://github.com/your-username/your-repo.git
```

Step 2: Make Changes and Push to GitHub

```
echo "Collaborator change" > collaborator.txt
git add collaborator.txt
git commit -m "Added collaborator file"
git push origin main
```

Step 3: Open a Pull Request (PR)

- Go to your GitHub repository.
- Click on "Pull Requests" > "New Pull Request" > Compare changes.
- Click "Create Pull Request" and add a description.
- Review and merge the PR.

Forking a Repository and Merging Changes

Step 1: Fork a Repository

- Go to a public GitHub repository.
- Click "Fork" (top-right corner).

Step 2: Clone the Forked Repository

```
git clone https://github.com/your-username/forked-repo.git
cd forked-repo
```

Step 3: Make Changes and Push to Your Forked Repository

```
echo "Forked repo modification" > forked.txt
git add forked.txt
git commit -m "Modified forked repo"
git push origin main
```

Step 4: Create a Pull Request to the Original Repository

- Go to the original repository.
- Click on "Pull Requests" > "New Pull Request" > Compare changes.
- Click "Create Pull Request" and submit it.

Step 5: Sync Forked Repository with the Original Repository

3. Creating a YAML files to define workflow configuration in github repository for CI/CD pipelining

Steps:

1. Create a GitHub Repository

- Go to [GitHub](#) and log in.
- Click on the **New Repository** button.
- Provide a name and description, and choose visibility (public/private).